

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 3**



BUILD A SCROLLABLE LIST

Oleh:

Abdurrahman Gilang Harjuna

NIM. 2310817110004

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN I
MODUL 2

Laporan Praktikum Pemrograman Mobile Modul 3: Build a Scrollable List ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Abdurrahman Gilang Harjuna
NIM : 2310817110004

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I
NIP. 19881027 201903 20 13

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1.....	6
A. Source Code	8
B. Output Program.....	16
C. Pembahasan	16
D. Tautan Git	20

DAFTAR GAMBAR

Gambar 1. Screenshot hasil soal 1.....	16
--	----

DAFTAR TABEL

Table 1. Source Code Jawaban Soal 1.....	8
Table 2. Source Code Jawaban Soal 1.....	9
Table 3. Source Code Jawaban Soal 1.....	10
Table 4. Source Code Jawaban Soal 1.....	11
Table 5. Source Code Jawaban Soal 1.....	12
Table 6. Source Code Jawaban Soal 1.....	13
Table 7. Source Code Jawaban Soal 11.....	14
Table 8. Source Code Jawaban Soal 1.....	15

SOAL 1

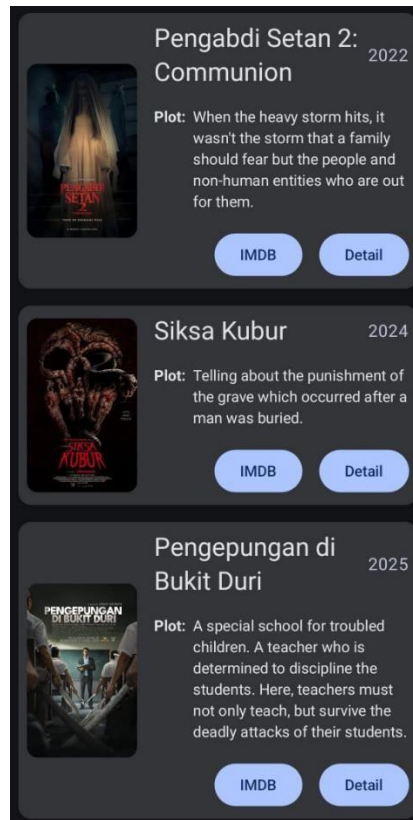
Soal Praktikum:

Buatlah sebuah aplikasi Android menggunakan XML atau Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) atau LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut:
 - a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
 - b. Button kedua menggunakan Navigation component/intent untuk membuka laman detail item
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
6. Saat orientasi perangkat berubah/dirotaasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
8. Aplikasi berbasis XML harus menggunakan ViewBinding

2. Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Dusahakan agar desain UI item list menyerupai UI berikut:



Gambar 1. Contoh UI List

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:



Gambar 2. Contoh UI Detail

A. Source Code

1. MainActivity.kt

```
1 package com.example.zennfit
2
3 import android.os.Bundle
4 import androidx.appcompat.app.AppCompatActivity
5 import com.example.zennfit.databinding.ActivityMainBinding
6
7 class MainActivity : AppCompatActivity() {
8     data class AlatGym(
9         val nama: String,
10        val descSingkat: String,
11        val deskripsi: String,
12        val gambarResId: Int,
13        val gifResId: Int
14    )
15
16    private lateinit var binding: ActivityMainBinding
17
18    override fun onCreate(savedInstanceState: Bundle?) {
19        super.onCreate(savedInstanceState)
20        binding = ActivityMainBinding.inflate(layoutInflater)
21        setContentView(binding.root)
22
23        // Tampilkan HomeFragment sebagai default
24        if (savedInstanceState == null) {
25            supportFragmentManager.beginTransaction()
26                .replace(R.id.fragment_container, HomeFragment())
27                .commit()
28        }
29    }
30 }
```

Table 1. Source Code Jawaban Soal 1

2. activity_main.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout
3     xmlns:android="http://schemas.android.com/apk/res/android"
4     android:layout_width="match_parent"
5     android:layout_height="match_parent"
6     android:orientation="vertical">
7
8     <TextView
9         android:layout_width="match_parent"
10        android:layout_height="wrap_content"
11        android:gravity="center"
12        android:padding="16dp"
13        android:text="ZennFit"
14        android:background="@color/black"
15        android:textColor="@android:color/white"
16        android:textSize="24sp" />
17
18    <FrameLayout
19        android:id="@+id/fragment_container"
20    />
21 />
```



```

        android:layout_width="match_parent"
        android:layout_height="match_parent" />
    </LinearLayout>

```

Table 2. Source Code Jawaban Soal 1

3. HomeFragment.kt

```

1 package com.example.zennfit
2
3 import android.os.Bundle
4 import android.view.LayoutInflater
5 import android.view.View
6 import android.view.ViewGroup
7 import androidx.fragment.app.Fragment
8 import androidx.recyclerview.widget.LinearLayoutManager
9 import com.example.zennfit.databinding.FragmentHomeBinding
10
11 class HomeFragment : Fragment() {
12
13     private var _binding: FragmentHomeBinding? = null
14     private val binding get() = _binding!!
15
16     // Gunakan data class dari MainActivity
17     private lateinit var alatList: List<MainActivity.AlatGym>
18
19     override fun onCreateView(
20         inflater: LayoutInflater, container: ViewGroup?,
21         savedInstanceState: Bundle?
22     ): View {
23         _binding = FragmentHomeBinding.inflate(inflater, container,
24 false)
25         return binding.root
26     }
27
28     override fun onViewCreated(view: View, savedInstanceState:
29 Bundle?) {
30         super.onViewCreated(view, savedInstanceState)
31
32         // Pindahkan pengisian alatList ke onViewCreated, karena
33 sudah ada konteks
34         val context = requireContext()
35         alatList = listOf(
36             MainActivity.AlatGym(
37                 "BenchPress",
38                 "Melatih otot dada dengan cara mendorong barbel dari
39 posisi telentang di bangku datar.",
40                 context.getString(R.string.desc_benchpress),
41                 R.drawable.benchpress,
42                 R.drawable.incline_benchpress
43             ),
44             MainActivity.AlatGym(
45                 "Row Cable",
46                 "Melatih otot punggung dengan gerakan mendayung dari
47 posisi duduk.",
48                 context.getString(R.string.desc_row),
49                 R.drawable.cablerow,

```

```

        R.drawable._seated
    ),
    MainActivity.AlatGym(
        "Shoulderpress Machine",
        "Melatih otot bahu dengan mendorong beban ke atas
dari posisi duduk.",
        context.getString(R.string.desc_shoulder),
        R.drawable.shoulderpress,
        R.drawable.shoulderpress_machine
    ),
    MainActivity.AlatGym(
        "Legpress Machine",
        "Melatih otot kaki dengan menekan beban menjauh
menggunakan kedua kaki.",
        context.getString(R.string.desc_legpress),
        R.drawable.legpress,
        R.drawable.legpressmachine
    ),
    MainActivity.AlatGym(
        "Fly Machine",
        "Melatih otot dada dengan gerakan merapatkan lengan
dari samping ke depan seperti memeluk.",
        context.getString(R.string.desc_fly),
        R.drawable.fly,
        R.drawable.fly_machine
    )
)

binding.recyclerView.layoutManager =
LinearLayoutManager(requireContext())
binding.recyclerView.adapter = AlatGymAdapter(alatList) {
    alat ->
        // Kirim data ke DetailFragment menggunakan Bundle
        val fragment = DetailFragment().apply {
            arguments = Bundle().apply {
                putString("nama", alat.nama)
                putString("deskripsi", alat.deskripsi)
                putInt("gambar", alat.gambarResId)
                putInt("gif", alat.gifResId)
            }
        }

        // Navigasi ke DetailFragment
        parentFragmentManager.beginTransaction()
            .replace(R.id.fragment_container, fragment)
            .addToBackStack(null)
            .commit()
    }
}

override fun onDestroyView() {
    super.onDestroyView()
    _binding = null
}
}

```

Table 3. Source Code Jawaban Soal 1

4. Fragment_home.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout
3   xmlns:android="http://schemas.android.com/apk/res/android"
4     android:layout_width="match_parent"
5     android:layout_height="match_parent"
6     android:orientation="vertical">
7
8     <androidx.recyclerview.widget.RecyclerView
9       android:id="@+id/recyclerView"
10      android:layout_width="match_parent"
11      android:layout_height="match_parent"
12      android:padding="8dp"
13      android:contentDescription="Daftar alat gym" />
14 </LinearLayout>
```

Table 4. Source Code Jawaban Soal 1

5. DetailFragment.kt

```
1 package com.example.zennfit
2
3 import android.os.Bundle
4 import android.view.LayoutInflater
5 import android.view.View
6 import android.view.ViewGroup
7 import androidx.fragment.app.Fragment
8 import com.bumptech.glide.Glide
9 import com.example.zennfit.databinding.FragmentDetailBinding
10
11 class DetailFragment : Fragment() {
12
13     private var _binding: FragmentDetailBinding? = null
14     private val binding get() = _binding!!
15
16     companion object {
17         fun newInstance(nama: String, deskripsi: String, gambar: Int,
18 gif: Int): DetailFragment {
19             val fragment = DetailFragment()
20             val bundle = Bundle().apply {
21                 putString("nama", nama)
22                 putString("deskripsi", deskripsi)
23                 putInt("gambar", gambar)
24                 putInt("gif", gif)
25             }
26             fragment.arguments = bundle
27             return fragment
28         }
29     }
30
31     override fun onCreateView(
32         inflater: LayoutInflater, container: ViewGroup?,
33         savedInstanceState: Bundle?
```

	<pre>): View { _binding = FragmentDetailBinding.inflate(inflater, container, false) return binding.root } override fun onViewCreated(view: View, savedInstanceState: Bundle?) { super.onViewCreated(view, savedInstanceState) val nama = arguments?.getString("nama") val deskripsi = arguments?.getString("deskripsi") val gambar = arguments?.getInt("gambar") ?: 0 val gif = arguments?.getInt("gif") ?: 0 activity?.title = nama binding.namaAlat.text = nama binding.deskripsiText.text = deskripsi binding.alatImage2.setImageResource(gambar) Glide.with(this).asGif().load(gif).into(binding.Gerakan) } override fun onDestroyView() { super.onDestroyView() _binding = null } } </pre>
--	--

Table 5. Source Code Jawaban Soal 1

6. Fragment_detail.xml

1	<?xml version="1.0" encoding="utf-8"?> <ScrollView
2	xmlns:android="http://schemas.android.com/apk/res/android"
3	android:layout_width="match_parent"
4	android:layout_height="match_parent">
5	<LinearLayout
6	android:layout_width="match_parent"
7	android:layout_height="wrap_content"
8	android:orientation="vertical"
9	android:padding="16dp">
10	<ImageView
11	android:id="@+id/alatImage2"
	android:layout_width="match_parent"
	android:layout_height="200dp"
	android:scaleType="centerCrop"
	android:contentDescription="Gambar Alat" />
	<TextView
	android:id="@+id/namaAlat"
	android:layout_width="match_parent"
	android:layout_height="wrap_content"
	android:text="Nama Alat"
	android:textSize="24sp"

	<pre> android:textStyle="bold" android:gravity="center" android:layout_marginTop="12dp" /> <TextView android:id="@+id/deskripsiText" android:layout_width="match_parent" android:layout_height="wrap_content" android:text="Deskripsi alat gym" android:textSize="16sp" android:layout_marginTop="12dp" /> <ImageView android:id="@+id/Gerakan" android:layout_width="match_parent" android:layout_height="200dp" android:layout_marginTop="16dp" android:scaleType="fitCenter" android:contentDescription="GIF Gerakan" /> </LinearLayout> </ScrollView> </pre>
--	--

Table 6. Source Code Jawaban Soal 1

7. AlatGymAdapter.kt

1	<code>package com.example.zennfit import android.view.LayoutInflater</code>
2	<code>import android.view.ViewGroup</code>
3	<code>import android.content.Intent</code>
4	<code>import android.net.Uri</code>
5	<code>import androidx.recyclerview.widget.RecyclerView</code>
6	<code>import com.example.zennfit.databinding.ListItemBinding</code>
7	<code>class AlatGymAdapter(</code>
8	<code> private val data: List<MainActivity.AlatGym>,</code>
9	<code> private val onClick: (MainActivity.AlatGym) -> Unit</code>
10	<code>) : RecyclerView.Adapter<AlatGymAdapter.ViewHolder>() {</code>
11	<code> class ViewHolder(val binding: ListItemBinding) :</code>
	<code>RecyclerView.ViewHolder(binding.root)</code>
	<code> override fun onCreateViewHolder(parent: ViewGroup, viewType:</code>
	<code>Int): ViewHolder {</code>
	<code> val binding =</code>
	<code>ListItemBinding.inflate(LayoutInflater.from(parent.context), parent,</code>
	<code>false)</code>
	<code> return ViewHolder(binding)</code>
	<code> }</code>
	<code> override fun onBindViewHolder(holder: ViewHolder, position: Int)</code>
	<code>{</code>
	<code> val alat = data[position]</code>
	<code> holder.binding.namaAlat.text = alat.nama</code>
	<code>holder.binding.alatImageList.setImageResource(alat.gambarResId)</code>
	<code> holder.binding.descSingkat.text = alat.descSingkat</code>

	<pre> holder.binding.viewButton.setOnClickListener { onClick(alat) } holder.binding.browserButton.setOnClickListener { val context = holder.itemView.context val intent = Intent(Intent.ACTION_VIEW, Uri.parse("https://www.google.com/search?q=\${Uri.encode(alat.nama)}")) context.startActivity(intent) } } override fun getItemCount(): Int = data.size } </pre>
--	---

Table 7. Source Code Jawaban Soal 11

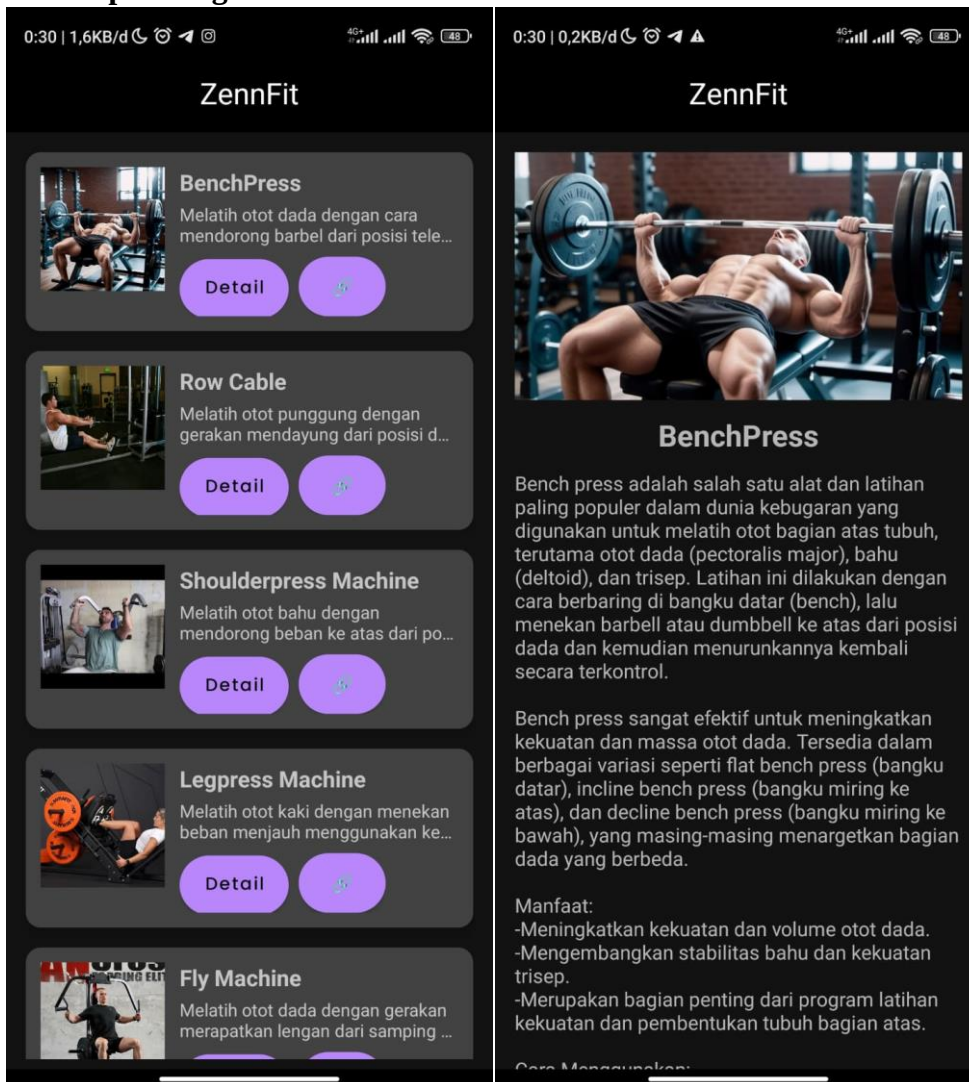
8. List_item.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.cardview.widget.CardView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:card_view="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="wrap_content"
7	android:layout_margin="8dp"
8	card_view:cardCornerRadius="12dp"
9	card_view:cardElevation="6dp">
10	<LinearLayout
11	android:layout_width="match_parent"
	android:layout_height="wrap_content"
	android:orientation="horizontal"
	android:padding="12dp">
	<ImageView
	android:id="@+id/alatImageList"
	android:layout_width="100dp"
	android:layout_height="100dp"
	android:scaleType="centerCrop"
	android:contentDescription="Gambar alat gymmm" />
	<LinearLayout
	android:layout_width="0dp"
	android:layout_weight="1"
	android:layout_height="wrap_content"
	android:orientation="vertical"
	android:layout_marginStart="12dp">
	<TextView
	android:id="@+id/namaAlat"
	android:layout_width="wrap_content"

	<pre> android:layout_height="wrap_content" android:text="Nama Alat" android:textSize="18sp" android:textStyle="bold" /> <TextView android:id="@+id/descSingkat" android:text="Deskripsi singkat alat di sini." android:textSize="14sp" android:layout_width="wrap_content" android:layout_height="wrap_content" android:layout_marginTop="4dp" android:maxLines="2" android:ellipsize="end" /> <LinearLayout android:layout_width="wrap_content" android:layout_height="wrap_content" android:orientation="horizontal"> <Button android:id="@+id/viewButton" android:text="Detail" android:layout_width="wrap_content" android:layout_height="wrap_content" android:layout_marginTop="8dp" android:layout_marginEnd="8dp" android:fontFamily="@font/poppins_medium" android:textAllCaps="false" android:background="@drawable/rounded_button"/> <Button android:id="@+id/browserButton" android:text="CS" android:layout_width="70dp" android:layout_height="wrap_content" android:layout_marginTop="8dp" android:background="@drawable/rounded_button"/> </LinearLayout> </LinearLayout> </LinearLayout> </androidx.cardview.widget.CardView> </pre>
--	---

Table 8. Source Code Jawaban Soal 1

B. Output Program



Gambar 1. Screenshot hasil soal 1

C. Pembahasan

1. MainActivity.kt:

package com.example.zennfit ini menyatakan bahwa file ini adalah bagian dari package com.example.zennfit. Selanjutnya ada import library, android.os.Bundle: Digunakan untuk menyimpan dan memulihkan state saat Activity dibuat. AppCompatActivity: Kelas dasar untuk Activity yang kompatibel dengan banyak versi Android. ActivityMainBinding: Kelas ViewBinding otomatis yang dihasilkan berdasarkan activity_main.xml. class MainActivity :

`AppCompatActivity()` { ini entry point dari aplikasi dan merupakan turunan dari `AppCompatActivity`.

Selanjutnya ada data class untuk merepresentasikan satu item alat gym. nama: Nama alat (misalnya "Bench Press"), descSingkat: Ringkasan deskripsi (misalnya "Alat melatih dada"), deskripsi: Penjelasan lengkap, gambarResId: ID resource gambar (misalnya `R.drawable.benchpress`), gifResId: ID resource untuk animasi atau gerakan alat (misalnya `R.drawable.benchpress_gif`). `private lateinit var binding: ActivityMainBinding` digunakan untuk mengakses komponen UI dalam `activity_main.xml` tanpa `findViewById`. `lateinit` ini artinya akan diinisialisasi nanti sebelum digunakan. pada kode `binding = ActivityMainBinding.inflate(layoutInflater)` `setContentView(binding.root)`, bagian ini `inflate(layoutInflater)` akan membuat objek binding dari layout `activity_main.xml`. `binding.root` adalah root view yang akan ditampilkan di layar. Selanjutnya menampilkan fragment default yang hanya dijalankan jika Activity baru pertama kali dibuat (bukan setelah rotasi layar). `supportFragmentManager` memulai `FragmentTransaction`: `replace(...)` akan mengganti isi `fragment_container` di `activity_main.xml` dengan `HomeFragment`. `commit()` menjalankan transaksinya.

2. `activity_main.xml`

`LinearLayout`: Mengatur elemen UI secara vertikal (dari atas ke bawah). `xmlns:android`: Deklarasi namespace untuk atribut Android. `layout_width="match_parent"` dan `layout_height="match_parent"`: Mengisi seluruh lebar dan tinggi layar. `orientation="vertical"`: Menyusun anak-anaknya secara vertikal.

Masuk keheader app, `layout_width="match_parent"`: Mengisi seluruh lebar layar. `layout_height="wrap_content"`: Tingginya disesuaikan dengan isi (teks). `gravity="center"`: Menempatkan teks di tengah horizontal. `padding="16dp"`: Memberi jarak di dalam komponen. `text="ZennFit"`: Menampilkan teks `ZennFit`. `background="@color/black"`: Latar belakang hitam (dari resource warna lokal). `textColor="@android:color/white"`: Teks berwarna putih. `textSize="24sp"`: Ukuran teks 24 scale-independent pixels.

Selanjutnya tempat untuk menampung fragment seperti `HomeFragment`, `DetailFragment`, dll. `id="@+id/fragment_container"`: Memberikan ID supaya dapat diakses dari kode Java/Kotlin. `layout_width` dan `layout_height` diatur ke `match_parent`, artinya mengisi sisa ruang layar setelah `TextView` di atas.

3. `HomeFragment.kt`

`class HomeFragment : Fragment()` turunan dari `Fragment`, yang artinya UI ini tidak berdiri sendiri, melainkan dimuat di dalam Activity (`MainActivity`). `Fragment` ini menampilkan list alat gym yang tersimpan dalam `RecyclerView`. `private var _binding: FragmentHomeBinding? = null` bagian `_binding` adalah variabel untuk mengakses komponen layout XML menggunakan `ViewBinding`. `FragmentHomeBinding` adalah binding otomatis yang dibuat dari `fragment_home.xml`. `private val binding get() = _binding!!` memastikan akses tidak null.

private lateinit var alatList: List<MainActivity.AlatGym> Menyimpan daftar data AlatGym yang akan ditampilkan. AlatGym adalah data class yang didefinisikan di MainActivity, merepresentasikan tiap alat gym (nama, deskripsi singkat, deskripsi panjang, gambar, dan gif). Di dalam onCreateView ada `_binding = FragmentHomeBinding.inflate(inflater, container, false)` return `binding.root` ini akan meng-inflate layout XML menjadi tampilan (View). Menggunakan ViewBinding untuk mengakses UI yang didefinisikan di `fragment_home.xml`.

Selanjutnya di dalam onCreateView disini membuat list berisi 5 alat gym. Setiap item berisi: nama: Nama alat, descSingkat: Deskripsi ringkas, deskripsi: Deskripsi panjang (diambil dari strings.xml), gambarResId: Gambar alat, gifResId: Gambar atau animasi tambahan. `binding.recyclerView.layoutManager = LinearLayoutManager(requireContext())` Mengatur RecyclerView supaya tampil secara vertikal (default) menggunakan LinearLayoutManager.

`binding.recyclerView.adapter = AlatGymAdapter(alatList) { alat -> ... }` Adapter dikustomisasi (AlatGymAdapter) untuk menampilkan data dan handle event klik pada tombol navigasi (ke detail). Callback `{ alat -> ... }` menangani klik navigasi ke detail. Lalu masuk navigasi ke DetailFragment saat diklik tombol navigasi maka data alat dikirim ke DetailFragment lewat Bundle menggunakan arguments. Selanjutnya Melakukan navigasi ke DetailFragment, mengganti fragment yang sedang tampil dan menambahkan ke backstack agar bisa kembali dengan tombol back. `onDestroyView()` ini mencegah memory leak dengan menghapus referensi ke binding saat fragment dihancurkan.

4. Fragment_Home.xml

`<LinearLayout ... >` ini adalah container yang menyusun elemen anaknya secara vertikal (atas ke bawah). `layout_width="match_parent"` dan `layout_height="match_parent"`: Layout ini akan mengisi seluruh layar pada lebar dan tinggi. `android:orientation="vertical"`: Elemen-elemen di dalam LinearLayout akan disusun secara vertikal.

`<androidx.recyclerview.widget.RecyclerView ... />` ini adalah komponen RecyclerView, digunakan untuk menampilkan daftar data yang dapat di-scroll secara efisien. `android:id="@+id/recyclerView"`: ID yang digunakan untuk mengakses RecyclerView dari kode Kotlin di HomeFragment. `layout_width="match_parent"` dan `layout_height="match_parent"`: RecyclerView akan mengisi seluruh ruang dari LinearLayout, baik lebar maupun tinggi. `android:padding="8dp"`: Memberi jarak 8dp di sekeliling RecyclerView, agar isi tidak mepet ke pinggir layar. `android:contentDescription="Daftar alat gym"`: Digunakan untuk aksesibilitas, agar pembaca layar bisa mengenali bahwa ini adalah daftar alat gym.

5. DetailFragment.kt

Didalam kode ini memakai `private var _binding: FragmentDetailBinding? = null`, menggunakan ViewBinding agar bisa mengakses elemen UI dari XML tanpa `findViewById`. `private val binding get() = _binding!!` `_binding` ini dibersihkan di `onDestroyView()` untuk

mencegah memory leak. Selanjutnya ada companion object yaitu membuat method newInstance() untuk mengemas data ke dalam Bundle dan mengatur sebagai arguments pada fragment. Memudahkan pemanggilan fragment ini dari luar (misalnya dari adapter atau fragment lain).

Menampilkan data alatnya, data diambil dari arguments (Bundle) yang dikirim sebelumnya saat berpindah dari list. Lalu ada set data ke UI, binding.namaAlat.text = nama, binding.deskripsiText.text = deskripsi, binding.alatImage2.setImageResource(gambar), Glide.with(this).asGif().load(gif).into(binding.Gerakan). nama dan deskripsi ditampilkan ke TextView. gambar ditampilkan ke ImageView. gif ditampilkan menggunakan Glide (asGif()), karena Android ImageView biasa tidak mendukung GIF secara default. Terakhir ada Membersihkan binding ini hanya praktik standar agar binding tidak menampung referensi view yang sudah hancur (mencegah memory leak).

6. Fragment_detail.xml

ScrollView: Agar tampilan bisa di-scroll, terutama jika konten lebih tinggi dari layar (penting saat orientasi berubah). LinearLayout: Digunakan untuk menyusun item secara vertikal (orientation="vertical"). ImageView gambar alatnya menampilkan gambar statis alat gym (misal: foto alat). scaleType="centerCrop": Memotong gambar agar memenuhi seluruh lebar, tetap menjaga rasio aspek. contentDescription: : Penting untuk aksesibilitas (dibaca oleh screen reader). TextView nama alatnya menampilkan nama alat gym (judul). Font besar (24sp), tebal, dan tengah (gravity="center") agar terlihat seperti header.

Selanjutnya ke TextView Deskripsi alatnya menampilkan deskripsi panjang alat gym. Lebih kecil dari judul (16sp) dan diletakkan setelah nama alat. ImageView untuk gif gambar menampilkan gerakan alat dalam bentuk GIF. scaleType="fitCenter" agar GIF ditampilkan penuh namun tetap proporsional. Dimuat menggunakan Glide di DetailFragment.kt.

7. AlatGymAdapter.kt

class AlatGymAdapter(private val data: List<MainActivity.AlatGym>, data: daftar item alat gym yang akan ditampilkan. private val onClick: (MainActivity.AlatGym) -> Unit) : RecyclerView.Adapter<AlatGymAdapter.ViewHolder>() {
 onClick: lambda function (callback) untuk meng-handle klik tombol detail.
 class ViewHolder(val binding: ListItemBinding) : RecyclerView.ViewHolder(binding.root) menyimpan binding ke tampilan item list (list_item.xml). Menggunakan ViewBinding agar lebih aman dan efisien saat mengakses komponen layout.
 val alat = data[position] mengambil item berdasarkan posisi dari List<AlatGym>. Lalu Menampilkan nama alat, gambar, dan deskripsi singkat di layout item list dengan binding.

holder.binding.viewButton.setOnClickListener { onClick(alat)} Saat tombol "Lihat" ditekan, memanggil fungsi onClick() dari constructor adapter. Diimplementasikan di HomeFragment untuk navigasi ke DetailFragment. Lalu ke button browser akan membuka

browser dan melakukan pencarian Google berdasarkan nama alat. override fun getItemCount(): Int = data.size akan mengembalikan jumlah item pada list.

8. List_item.xml

```
<androidx.cardview.widget.CardView,  
android:layout_margin="8dp" card_view:cardCornerRadius="12dp"  
card_view:cardElevation="6dp">
```

membuat efek kartu mengambang (elevasi) dengan sudut bulat. Memberi kesan UI modern dan rapi. pada linera layout, android:orientation="horizontal" membuat isi berjejer ke kanan: Gambar di kiri, teks dan tombol di kanan. ImageView alatImageView Menampilkan gambar alat gym. centerCrop: gambar dipotong agar pas dan tidak melebar/mengecil secara tidak proporsional. TextView namaAlat menampilkan nama alat gym. Teks tebal dan besar agar terlihat seperti judul. TextView DescSingkat menampilkan deskripsi singkat alat gym. Jika terlalu panjang, hanya ditampilkan 2 baris dan dipotong dengan Tombol detail saat ditekan, akan membuka DetailFragment. Memakai background kustom rounded_button agar tampil modern (biasanya berbentuk bulat dengan warna khusus). Tombol browser saat ditekan, akan membuka Google search dengan nama alat gym. Tombol lebih pendek (70dp) dan hanya berisi emoji/link icon. saya juga nambahkan styling di @font/poppins_medium: font khusus agar teks terlihat lebih profesional. @drawable/rounded_button: file drawable kustom yang biasanya digunakan untuk membuat tombol dengan sudut bulat dan warna tertentu.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/Specter735/Pemrograman-Mobile.git>